

08b_forward_v2_best_models_pipeline

April 28, 2026

1 08b - Forward Model v2 Best Models Pipeline

This notebook refines the forward prediction solution for competition scoring.

Previous result: - Notebook 07 showed that advanced v2 features improve fragmentation targets.
- Notebook 08 created a clean sklearn pipeline, but it used ExtraTrees for all targets. - Earlier boosting experiments showed that CatBoost was slightly better for `oversize_frac`.

This notebook builds a **best-of-both-worlds target-specific pipeline**:

Target	Model	Feature version
P80	ExtraTrees	v2 fragmentation features
fines_frac	ExtraTrees	v2 fragmentation features
oversize_frac	CatBoost if available, otherwise ExtraTrees	tested/selected feature set
R95	ExtraTrees	v1 distance features
R50_fines	ExtraTrees	v1 distance features
R50_oversize	ExtraTrees	v1 distance features

Main outputs:

outputs/submissions/prediction_submission_forward_v2_best_models.csv
outputs/models/final_forward_v2_best_models_pipeline.joblib
outputs/models/final_forward_v2_best_models_metadata.json

```
[1]: from pathlib import Path
import json
import warnings
import joblib
import numpy as np
import pandas as pd

from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import ExtraTreesRegressor

warnings.filterwarnings("ignore")
```

```

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
    cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for path in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    path.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward dir:", FORWARD_DIR)
print("Submissions dir:", SUBMISSIONS_DIR)
print("Models dir:", MODELS_DIR)

```

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/forward_prediction
Submissions dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions
Models dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models

```

1.1 1. Load data and constraints

```

[2]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor",
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
raw_test = pd.read_csv(FORWARD_DIR / "test.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

```

```

constraints_path = INVERSE_DIR / "constraints.json"
if constraints_path.exists():
    with open(constraints_path, "r") as f:
        constraints_data = json.load(f)
        output_constraints = constraints_data["constraints"]
else:
    output_constraints = {"p80_min": 96.0, "p80_max": 101.0, "r95_max": 175.0}

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]

print("Raw train:", raw_train.shape)
print("Raw test:", raw_test.shape)
print("Targets:", y.shape)
display(pd.DataFrame([output_constraints]))
display(raw_train.head())
display(y.head())

```

Raw train: (2930, 8)

Raw test: (492, 8)

Targets: (2930, 6)

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	shape_factor
0	3.826405	0.818303	0.861258	1.305809	0.337215	3.71	0.781263	0.784028
1	2.828754	1.193036	0.561245	3.494501	0.058029	1.62	0.136205	0.922737
2	3.068907	0.605872	0.948860	1.366386	0.315632	3.71	0.774704	0.954922
3	2.700574	1.073708	0.713705	3.599419	0.033062	1.62	0.144204	0.932911
4	3.484022	0.863568	1.237205	1.996742	0.278207	9.81	0.414620	1.260855

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	76.972350	0.184728	0.016671	198.938699	175.527939	76.235779
1	269.057465	0.000622	0.916734	239.268477	447.157838	141.894047
2	104.070923	0.070343	0.094438	192.986417	189.286407	84.235774
3	257.618403	0.001026	0.880122	289.289693	500.000028	169.866473
4	111.717167	0.058576	0.136166	94.229304	97.614864	42.928393

1.2 2. Sklearn-compatible feature engineering

We keep the pipeline explicit:

```

Pipeline([
    ("features", PhysicsFeatureEngineer(use_advanced=True/False)),
    ("select", ColumnSelector(feature_list)),
    ("model", model),
])

```

```

[3]: class PhysicsFeatureEngineer(BaseEstimator, TransformerMixin):
    def __init__(self, use_advanced: bool = False):
        self.use_advanced = use_advanced

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = pd.DataFrame(X).copy()
        X = X[input_cols].copy()
        eps = 1e-9

        # 1. Energy transfer features
        X["effective_energy"] = X["energy"] * X["coupling"]
        X["log_energy"] = np.log1p(X["energy"])
        X["log_effective_energy"] = np.log1p(X["effective_energy"])

        # 2. Angle decomposition
        X["sin_angle"] = np.sin(X["angle_rad"])
        X["cos_angle"] = np.cos(X["angle_rad"])
        X["tan_angle"] = np.tan(X["angle_rad"])
        X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
        X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
        X["vertical_horizontal_ratio"] = X["vertical_energy"] /  $\sqrt{X["horizontal_energy"] + eps}$ 

        # 3. Material / fragmentation proxies
        X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
        X["effective_energy_per_strength"] = X["effective_energy"] /  $\sqrt{X["strength"] + eps}$ 
        X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
        X["fragmentation_index"] = X["effective_energy"] * X["porosity"] /  $\sqrt{X["strength"] + eps}$ 
        X["coupling_porosity"] = X["coupling"] * X["porosity"]
        X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
        X["porosity_strength"] = X["porosity"] * X["strength"]

        # 4. Gravity and range proxies
        X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
        X["effective_energy_per_gravity"] = X["effective_energy"] /  $\sqrt{X["gravity"] + eps}$ 

```

```

X["horizontal_energy_per_gravity"] = X["horizontal_energy"] / (
↪X["gravity"] + eps)
X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"] +
↪eps)

# 5. Atmosphere and drag proxies
X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *
↪X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

# 6. Pi-like scaling proxies
X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] +
↪eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"] +
↪eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] *
↪X["coupling"] + eps)

# 7. Regime indicators
X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2
    + X["atm_regime"]
)

# 8. Cross-regime proxies
X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪sqrt(X["gravity"]) + eps)
X["fragility"] = X["porosity"] / (X["strength"] + eps)
X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) /
↪(X["gravity"] * X["strength"] + eps)
X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]
X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] /
↪(X["energy"] + eps)

if self.use_advanced:

```

```

        # Advanced v2 features selected after notebook 07.
        X["froude_proxy"] = np.sqrt(X["effective_energy"] / (X["gravity"] +
        ↪eps))

        X["stress_ratio_compact"] = X["effective_energy"] /
        ↪(X["material_resistance_index"] + eps)

        X["sqrt_effective_energy_per_strength"] = np.
        ↪sqrt(X["effective_energy_per_strength"].clip(lower=0))

        X["sqrt_effective_energy_per_gravity"] = np.
        ↪sqrt(X["effective_energy_per_gravity"].clip(lower=0))

        return X

class ColumnSelector(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = list(columns)

    def fit(self, X, y=None):
        missing = [c for c in self.columns if c not in X.columns]
        if missing:
            raise ValueError(f"Missing columns: {missing}")
        return self

    def transform(self, X):
        return X[self.columns].copy()

```

1.3 3. Feature sets and model builders

```

[4]: raw_features = input_cols.copy()

fragmentation_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪"regime_combo",
]

fragmentation_features_v2 = fragmentation_features_v1 + [
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
]

```

```

# Extended-plus-regimes is used for CatBoost oversize because it performed
↳slightly better in previous experiments.
extended_plus_regimes_features = raw_features + [
    "effective_energy", "log_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "tan_angle",
    "horizontal_energy", "vertical_energy", "vertical_horizontal_ratio",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↳"atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_strength_proxy", "pi_atmosphere_proxy",
    "scaled_energy", "fragility", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "coupling_per_atm_clipped", "log_coupling_per_atm", "retention_factor",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↳"regime_combo",
]

extended_plus_regimes_features_v2 = extended_plus_regimes_features + [
    "froude_proxy",
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
    "sqrt_effective_energy_per_gravity",
]

distance_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↳"atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↳"regime_combo",
]

def build_extratrees(random_state=42):
    return ExtraTreesRegressor(
        n_estimators=800,
        max_features="sqrt",

```

```

        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

try:
    from catboost import CatBoostRegressor

    def build_catboost(random_state=42):
        return CatBoostRegressor(
            iterations=1200,
            learning_rate=0.03,
            depth=6,
            l2_leaf_reg=5.0,
            loss_function="RMSE",
            random_seed=random_state,
            verbose=False,
        )

    CATBOOST_AVAILABLE = True
except Exception as e:
    print("CatBoost unavailable. Falling back to ExtraTrees for oversize_frac.")
    print("Reason:", e)
    CATBOOST_AVAILABLE = False

print("CatBoost available:", CATBOOST_AVAILABLE)

```

CatBoost available: True

1.4 4. Candidate configurations

We compare a small number of reasonable target-specific configurations, not dozens of random experiments. This is a competition-focused refinement step.

```

[6]: # Baseline hybrid from notebook 08.
config_08_hybrid = {
    "P80": {"model": "ExtraTrees", "feature_version": "v2", "features":
    ↪fragmentation_features_v2},
    "fines_frac": {"model": "ExtraTrees", "feature_version": "v2", "features":
    ↪fragmentation_features_v2},
    "oversize_frac": {"model": "ExtraTrees", "feature_version": "v2",
    ↪"features": fragmentation_features_v2},
    "R95": {"model": "ExtraTrees", "feature_version": "v1", "features":
    ↪distance_features_v1},
    "R50_fines": {"model": "ExtraTrees", "feature_version": "v1", "features":
    ↪distance_features_v1},

```

```

    "R50_oversize": {"model": "ExtraTrees", "feature_version": "v1", "features":
↳ distance_features_v1},
}

# Best-model refinement: use CatBoost for oversize_frac if available.
config_best_models = {
    "P80": {"model": "ExtraTrees", "feature_version": "v2", "features":↳
↳ fragmentation_features_v2},
    "fines_frac": {"model": "ExtraTrees", "feature_version": "v2", "features":↳
↳ fragmentation_features_v2},
    "oversize_frac": {
        "model": "CatBoost" if CATBOOST_AVAILABLE else "ExtraTrees",
        "feature_version": "v2" if CATBOOST_AVAILABLE else "v2",
        "features": extended_plus_regimes_features_v2 if CATBOOST_AVAILABLE↳
↳ else fragmentation_features_v2,
    },
    "R95": {"model": "ExtraTrees", "feature_version": "v1", "features":↳
↳ distance_features_v1},
    "R50_fines": {"model": "ExtraTrees", "feature_version": "v1", "features":↳
↳ distance_features_v1},
    "R50_oversize": {"model": "ExtraTrees", "feature_version": "v1", "features":
↳ distance_features_v1},
}

configs = {
    "08_hybrid_extra_trees_only": config_08_hybrid,
    "08b_best_models": config_best_models,
}

for config_name, cfg in configs.items():
    print("", config_name)
    display(pd.DataFrame([
        {
            "target": t,
            "model": c["model"],
            "feature_version": c["feature_version"],
            "n_features": len(c["features"]),
        }
        for t, c in cfg.items()
    ]))

```

08_hybrid_extra_trees_only

	target	model	feature_version	n_features
0	P80	ExtraTrees	v2	36
1	fines_frac	ExtraTrees	v2	36
2	oversize_frac	ExtraTrees	v2	36
3	R95	ExtraTrees	v1	33

4	R50_fines	ExtraTrees	v1	33
5	R50_oversize	ExtraTrees	v1	33

08b_best_models

	target	model	feature_version	n_features
0	P80	ExtraTrees	v2	36
1	fines_frac	ExtraTrees	v2	36
2	oversize_frac	CatBoost	v2	52
3	R95	ExtraTrees	v1	33
4	R50_fines	ExtraTrees	v1	33
5	R50_oversize	ExtraTrees	v1	33

1.5 5. Pipeline wrapper

```
[7]: def get_model_builder(model_name):
    if model_name == "ExtraTrees":
        return build_extratrees
    if model_name == "CatBoost" and CATBOOST_AVAILABLE:
        return build_catboost
    return build_extratrees

def build_target_pipeline_from_config(target, config, random_state=42):
    cfg = config[target]
    use_advanced = cfg["feature_version"] == "v2"
    model_builder = get_model_builder(cfg["model"])
    model = model_builder(random_state=random_state)
    return Pipeline(steps=[
        ("features", PhysicsFeatureEngineer(use_advanced=use_advanced)),
        ("select", ColumnSelector(cfg["features"])),
        ("model", model),
    ])

def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
        return np.clip(preds, 0, 1)
    return np.clip(preds, 0, None)

class TargetSpecificPipelineModel:
    def __init__(self, target_config, random_state=42):
        self.target_config = target_config
        self.random_state = random_state
        self.pipelines_ = {}
```

```

def fit(self, X_raw, y_df):
    for i, target in enumerate(target_cols):
        pipe = build_target_pipeline_from_config(target, self.
target_config, random_state=self.random_state + i)
        pipe.fit(X_raw, y_df[target])
        self.pipelines_[target] = pipe
    return self

def predict(self, X_raw):
    preds = pd.DataFrame(index=X_raw.index)
    for target in target_cols:
        pred = self.pipelines_[target].predict(X_raw)
        preds[target] = clip_predictions(pred, target)
    return preds[target_cols]

def describe(self):
    rows = []
    for target, pipe in self.pipelines_.items():
        cfg = self.target_config[target]
        rows.append({
            "target": target,
            "model": cfg["model"],
            "feature_version": cfg["feature_version"],
            "n_features": len(cfg["features"]),
            "pipeline_model": type(pipe.named_steps["model"]).__name__,
        })
    return pd.DataFrame(rows)

```

1.6 6. Metrics

```

[8]: def regression_metrics(y_true, y_pred, target_name=None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    abs_error = np.abs(y_pred - y_true)
    out = {
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2,
        "Median_AE": np.median(abs_error),
        "P90_AE": np.percentile(abs_error, 90),
        "P95_AE": np.percentile(abs_error, 95),
        "Max_AE": np.max(abs_error),
        "Bias": float(np.mean(y_pred - y_true)),
    }

```

```

if target_name is not None:
    out["normalized_MAE"] = mae / (y[target_name].std() + 1e-9)
    out["normalized_RMSE"] = rmse / (y[target_name].std() + 1e-9)
return out

def feasibility_mask(df_targets):
    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]_
    ↪ <= r95_max)

def custom_constraint_metrics(y_true_df, y_pred_df):
    true_feasible = feasibility_mask(y_true_df)
    pred_feasible = feasibility_mask(y_pred_df)
    tp = int((true_feasible & pred_feasible).sum())
    fp = int((~true_feasible & pred_feasible).sum())
    fn = int((true_feasible & ~pred_feasible).sum())
    precision = tp / (tp + fp + 1e-9)
    recall = tp / (tp + fn + 1e-9)
    f1 = 2 * precision * recall / (precision + recall + 1e-9)
    near_zone = (y_true_df["P80"].between(80, 120)) & (y_true_df["R95"] <= 250)
    out = {
        "n_true_feasible": int(true_feasible.sum()),
        "n_pred_feasible": int(pred_feasible.sum()),
        "true_positive": tp,
        "false_positive": fp,
        "false_negative": fn,
        "feasible_precision": precision,
        "feasible_recall": recall,
        "feasible_f1": f1,
        "near_zone_count": int(near_zone.sum()),
    }
    if near_zone.sum() > 0:
        out["near_zone_MAE_P80"] = mean_absolute_error(y_true_df.loc[near_zone,
    ↪ "P80"], y_pred_df.loc[near_zone, "P80"])
        out["near_zone_MAE_R95"] = mean_absolute_error(y_true_df.loc[near_zone,
    ↪ "R95"], y_pred_df.loc[near_zone, "R95"])
        out["near_zone_R95_bias"] = float((y_pred_df.loc[near_zone, "R95"] -
    ↪ y_true_df.loc[near_zone, "R95"]).mean())
    return out

```

1.7 7. Cross-validation comparison

```

[9]: def cross_validate_target_specific_config(config_name, target_config, X_raw,
    ↪ y_df, n_splits=5, random_state=42):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    oof = pd.DataFrame(index=y_df.index, columns=target_cols, dtype=float)

```

```

fold_rows = []

for fold, (train_idx, valid_idx) in enumerate(kf.split(X_raw), start=1):
    print(f"{config_name} | Fold {fold}/{n_splits}")
    X_train = X_raw.iloc[train_idx].reset_index(drop=True)
    X_valid = X_raw.iloc[valid_idx].reset_index(drop=True)
    y_train = y_df.iloc[train_idx].reset_index(drop=True)
    y_valid = y_df.iloc[valid_idx].reset_index(drop=True)

    model = TargetSpecificPipelineModel(target_config=target_config,
    ↪random_state=random_state + fold)
    model.fit(X_train, y_train)
    pred_valid = model.predict(X_valid)
    pred_valid.index = valid_idx
    oof.loc[valid_idx, target_cols] = pred_valid[target_cols]

    for target in target_cols:
        metrics = regression_metrics(y_valid[target], pred_valid[target],
    ↪target_name=target)
        metrics.update({
            "fold": fold,
            "target": target,
            "config": config_name,
            "model": target_config[target]["model"],
            "feature_version": target_config[target]["feature_version"],
            "n_features": len(target_config[target]["features"]),
        })
        fold_rows.append(metrics)

overall_rows = []
for target in target_cols:
    metrics = regression_metrics(y_df[target], oof[target],
    ↪target_name=target)
    metrics.update({
        "fold": "OOF",
        "target": target,
        "config": config_name,
        "model": target_config[target]["model"],
        "feature_version": target_config[target]["feature_version"],
        "n_features": len(target_config[target]["features"]),
    })
    overall_rows.append(metrics)

overall_df = pd.DataFrame(overall_rows)
fold_df = pd.DataFrame(fold_rows)
constraint_df = pd.DataFrame([custom_constraint_metrics(y_df, oof)])
constraint_df["config"] = config_name

```

```

    return overall_df, fold_df, oof, constraint_df

RUN_COMPARISON_CV = True

if RUN_COMPARISON_CV:
    all_overall = []
    all_folds = []
    all_constraints = []
    oof_store = {}

    for config_name, cfg in configs.items():
        overall_df, fold_df, oof_df, constraint_df =
↪ cross_validate_target_specific_config(
            config_name=config_name,
            target_config=cfg,
            X_raw=raw_train,
            y_df=y,
            n_splits=5,
            random_state=42,
        )
        all_overall.append(overall_df)
        all_folds.append(fold_df)
        all_constraints.append(constraint_df)
        oof_store[config_name] = oof_df

    comparison_results = pd.concat(all_overall, ignore_index=True)
    fold_results = pd.concat(all_folds, ignore_index=True)
    constraint_comparison = pd.concat(all_constraints, ignore_index=True)

    display(comparison_results.sort_values(["target", "normalized_MAE"]))
    display(constraint_comparison.sort_values("feasible_f1", ascending=False))
else:
    print("Set RUN_COMPARISON_CV=True to run the comparison.")

```

```

08_hybrid_extra_trees_only | Fold 1/5
08_hybrid_extra_trees_only | Fold 2/5
08_hybrid_extra_trees_only | Fold 3/5
08_hybrid_extra_trees_only | Fold 4/5
08_hybrid_extra_trees_only | Fold 5/5
08b_best_models | Fold 1/5
08b_best_models | Fold 2/5
08b_best_models | Fold 3/5
08b_best_models | Fold 4/5
08b_best_models | Fold 5/5

```

	MAE	RMSE	R2	Median_AE	P90_AE	P95_AE	
↪ Max_AE	Bias	normalized_MAE	normalized_RMSE	fold	target		↪
↪	config	\					

0	7.649464	10.159581	0.976120	5.889286	16.804011	21.487673	53.	
	↪412408	-0.005756	0.116332		0.154506	00F	P80	↪
	↪08_hybrid_extra_trees_only							
6	7.649464	10.159581	0.976120	5.889286	16.804011	21.487673	53.	
	↪412408	-0.005756	0.116332		0.154506	00F	P80	↪
	↪08b_best_models							
4	50.393094	78.496171	0.898887	27.455684	129.306490	170.647824	533.	
	↪891646	-0.386028	0.204104		0.317929	00F	R50_fines	↪
	↪08_hybrid_extra_trees_only							
10	50.393094	78.496171	0.898887	27.455684	129.306490	170.647824	533.	
	↪891646	-0.386028	0.204104		0.317929	00F	R50_fines	↪
	↪08b_best_models							
5	22.731340	38.575224	0.873407	11.680371	56.905391	81.580273	346.	
	↪724231	-0.000924	0.209627		0.355739	00F	R50_oversize	↪
	↪08_hybrid_extra_trees_only							
11	22.731340	38.575224	0.873407	11.680371	56.905391	81.580273	346.	
	↪724231	-0.000924	0.209627		0.355739	00F	R50_oversize	↪
	↪08b_best_models							
3	41.432050	68.507429	0.917493	20.851114	103.401705	149.134690	471.	
	↪617395	-0.366155	0.173688		0.287191	00F	R95	↪
	↪08_hybrid_extra_trees_only							
9	41.432050	68.507429	0.917493	20.851114	103.401705	149.134690	471.	
	↪617395	-0.366155	0.173688		0.287191	00F	R95	↪
	↪08b_best_models							
1	0.006317	0.013826	0.959368	0.000613	0.019765	0.031609	0.	
	↪141095	-0.000035	0.092084		0.201540	00F	fines_frac	↪
	↪08_hybrid_extra_trees_only							
7	0.006317	0.013826	0.959368	0.000613	0.019765	0.031609	0.	
	↪141095	-0.000035	0.092084		0.201540	00F	fines_frac	↪
	↪08b_best_models							
2	0.025680	0.035189	0.990552	0.018655	0.057401	0.075011	0.	
	↪167200	-0.000040	0.070923		0.097185	00F	oversize_frac	↪
	↪08_hybrid_extra_trees_only							
8	0.025906	0.035514	0.990377	0.018795	0.058452	0.076274	0.	
	↪181910	0.000375	0.071547		0.098081	00F	oversize_frac	↪
	↪08b_best_models							

	model	feature_version	n_features
0	ExtraTrees	v2	36
6	ExtraTrees	v2	36
4	ExtraTrees	v1	33
10	ExtraTrees	v1	33
5	ExtraTrees	v1	33
11	ExtraTrees	v1	33
3	ExtraTrees	v1	33
9	ExtraTrees	v1	33
1	ExtraTrees	v2	36

```

7  ExtraTrees          v2          36
2  ExtraTrees          v2          36
8   CatBoost          v2          52

   n_true_feasible  n_pred_feasible  true_positive  false_positive  \
↪false_negative  feasible_precision  feasible_recall  feasible_f1  \
↪near_zone_count  near_zone_MAE_P80  \
0           35           37           13           24           372
↪22           0.351351           0.371429           0.361111
↪ 4.584673
1           35           37           13           24           372
↪22           0.351351           0.371429           0.361111
↪ 4.584673

   near_zone_MAE_R95  near_zone_R95_bias  config
0           25.522959           13.911613  08_hybrid_extra_trees_only
1           25.522959           13.911613           08b_best_models

```

1.8 8. Choose final configuration

```

[10]: if RUN_COMPARISON_CV:
    best_by_target = comparison_results.sort_values(["target", "MAE"]).
    ↪groupby("target", as_index=False).first()
    print("Best by MAE")
    display(best_by_target[["target", "config", "model", "feature_version",
    ↪"MAE", "RMSE", "R2", "P95_AE", "n_features"]])

    pivot = comparison_results.pivot_table(index="target", columns="config",
    ↪values=["MAE", "RMSE", "P95_AE"], aggfunc="first")
    display(pivot)

    # Competition-oriented final choice.
    # If CatBoost is available, the best_models config is expected to keep the
    ↪CatBoost gain on oversize_frac.
    FINAL_CONFIG_NAME = "08b_best_models" if CATBOOST_AVAILABLE else
    ↪"08_hybrid_extra_trees_only"
    FINAL_CONFIG = configs[FINAL_CONFIG_NAME]
    print("Selected final config:", FINAL_CONFIG_NAME)

```

Best by MAE

```

      target          config  model feature_version
↪MAE      RMSE      R2      P95_AE  n_features
0      P80  08_hybrid_extra_trees_only  ExtraTrees      v2  7.
↪649464  10.159581  0.976120  21.487673      36
1      R50_fines  08_hybrid_extra_trees_only  ExtraTrees      v1  50.
↪393094  78.496171  0.898887  170.647824      33

```

2	R50_oversize	08_hybrid_extra_trees_only	ExtraTrees	v1	22.
	↪731340	38.575224	0.873407	81.580273	33
3	R95	08_hybrid_extra_trees_only	ExtraTrees	v1	41.
	↪432050	68.507429	0.917493	149.134690	33
4	finest_frac	08_hybrid_extra_trees_only	ExtraTrees	v2	0.
	↪006317	0.013826	0.959368	0.031609	36
5	oversize_frac	08_hybrid_extra_trees_only	ExtraTrees	v2	0.
	↪025680	0.035189	0.990552	0.075011	36

MAE

↪P95_AE

RMSE

config 08_hybrid_extra_trees_only 08b_best_models
 ↪08_hybrid_extra_trees_only 08b_best_models 08_hybrid_extra_trees_only
 ↪08b_best_models

target

P80		7.649464	7.649464	21.
	↪487673	21.487673	10.159581	10.159581
R50_fines		50.393094	50.393094	170.
	↪647824	170.647824	78.496171	78.496171
R50_oversize		22.731340	22.731340	81.
	↪580273	81.580273	38.575224	38.575224
R95		41.432050	41.432050	149.
	↪134690	149.134690	68.507429	68.507429
finest_frac		0.006317	0.006317	0.
	↪031609	0.031609	0.013826	0.013826
oversize_frac		0.025680	0.025906	0.
	↪075011	0.076274	0.035189	0.035514

Selected final config: 08b_best_models

1.9 9. Train final model and generate submission

```
[11]: final_model = TargetSpecificPipelineModel(target_config=FINAL_CONFIG,
↪random_state=42)
final_model.fit(raw_train, y)

display(final_model.describe())

test_predictions = final_model.predict(raw_test)
display(test_predictions.head())
display(test_predictions.describe().T)

submission = pd.DataFrame({"scenario_id": np.arange(len(raw_test))})
for col in target_cols:
    submission[col] = test_predictions[col].values
submission = submission[["scenario_id"] + target_cols]
```

```

submission_path = SUBMISSIONS_DIR /
    ↪ "prediction_submission_forward_v2_best_models.csv"
submission.to_csv(submission_path, index=False)

print("Saved submission to:", submission_path)
print("Shape:", submission.shape)
display(submission.head())
display(submission.tail())

```

	target	model	feature_version	n_features	pipeline_model
0	P80	ExtraTrees	v2	36	ExtraTreesRegressor
1	fines_frac	ExtraTrees	v2	36	ExtraTreesRegressor
2	oversize_frac	CatBoost	v2	52	CatBoostRegressor
3	R95	ExtraTrees	v1	33	ExtraTreesRegressor
4	R50_fines	ExtraTrees	v1	33	ExtraTreesRegressor
5	R50_oversize	ExtraTrees	v1	33	ExtraTreesRegressor

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	120.852192	0.120046	0.162922	726.196054	704.966598	312.487130
1	113.668032	0.048219	0.155590	204.758075	220.793490	102.212871
2	149.740914	0.121343	0.133007	809.085123	808.873827	348.096898
3	162.227452	0.008732	0.563936	531.683980	592.186388	277.513349
4	137.978607	0.047156	0.199129	197.063393	243.395743	100.912752

	count	mean	std	min	25%	50%
↪ 75%		max				
P80	492.0	157.708072	33.479974	76.016131	130.253486	153.137936
↪ 182.044124	230.519576					
fines_frac	492.0	0.050271	0.059135	0.001579	0.005592	0.017260
↪ 0.085643	0.265903					
oversize_frac	492.0	0.422195	0.238924	0.030346	0.200395	0.400134
↪ 0.663297	0.796151					
R95	492.0	298.530443	241.158254	48.776910	109.611058	177.453510
↪ 508.553148	996.810828					
R50_fines	492.0	330.556280	235.993585	64.210862	142.579155	213.098175
↪ 580.026235	873.066616					
R50_oversize	492.0	146.295647	108.429543	28.072745	58.843228	91.697836
↪ 263.430771	414.330461					

Saved submission to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/prediction_submission_forward_v2_best_models.csv
Shape: (492, 7)

	scenario_id	P80	fines_frac	oversize_frac	R95	R50_fines
↪ R50_oversize						
0	0	120.852192	0.120046	0.162922	726.196054	704.966598
↪ 312.487130						
1	1	113.668032	0.048219	0.155590	204.758075	220.793490
↪ 102.212871						

2	2	149.740914	0.121343	0.133007	809.085123	808.873827	↵
↵		348.096898					
3	3	162.227452	0.008732	0.563936	531.683980	592.186388	↵
↵		277.513349					
4	4	137.978607	0.047156	0.199129	197.063393	243.395743	↵
↵		100.912752					
	scenario_id	P80	finest_frac	oversize_frac	R95	R50_fines	↵
↵	R50_oversize						
487	487	152.206678	0.085654	0.200659	91.787908	109.365615	↵
↵		44.027042					
488	488	137.972752	0.022555	0.360027	581.289278	628.330345	↵
↵		287.685770					
489	489	166.438766	0.006598	0.543783	57.146187	70.314348	↵
↵		30.894136					
490	490	194.291398	0.003813	0.730864	152.386776	201.155009	↵
↵		86.035565					
491	491	207.066582	0.002437	0.709878	372.573623	507.461436	↵
↵		216.989868					

1.10 10. Compare test predictions with v1 and notebook 08 hybrid

```
[12]: comparison_files = {
    "v1_target_specific": SUBMISSIONS_DIR / ↵
    ↵ "prediction_submission_final_target_specific.csv",
    "v2_hybrid_extra_trees": SUBMISSIONS_DIR / ↵
    ↵ "prediction_submission_forward_v2_hybrid.csv",
    "v2_best_models": submission_path,
}

loaded = {}
for name, path in comparison_files.items():
    if path.exists():
        loaded[name] = pd.read_csv(path)
        print(name, path, loaded[name].shape)
    else:
        print("Missing", name, path)

rows = []
if "v2_best_models" in loaded:
    ref = loaded["v2_best_models"]
    for name, df in loaded.items():
        for target in target_cols:
            rows.append({
                "file": name,
                "target": target,
                "mean": df[target].mean(),
```

```

        "std": df[target].std(),
        "min": df[target].min(),
        "max": df[target].max(),
        "median": df[target].median(),
    })
test_distribution_comparison = pd.DataFrame(rows)
display(test_distribution_comparison)

if "v1_target_specific" in loaded:
    diff_rows = []
    v1 = loaded["v1_target_specific"]
    for target in target_cols:
        diff = ref[target] - v1[target]
        diff_rows.append({
            "target": target,
            "mean_diff_best_minus_v1": diff.mean(),
            "median_abs_diff": diff.abs().median(),
            "max_abs_diff": diff.abs().max(),
        })
    test_prediction_diff_vs_v1 = pd.DataFrame(diff_rows)
    display(test_prediction_diff_vs_v1)

```

v1_target_specific /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/prediction_submission_final_target_specific.csv (492, 7)

v2_hybrid_extra_trees /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/prediction_submission_forward_v2_hybrid.csv (492, 7)

v2_best_models /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/prediction_submission_forward_v2_best_models.csv (492, 7)

	file	target	mean	std	min	max	median
0	v1_target_specific	P80	154.410581	29.205843	77.932694	219.854089	152.472220
1	v1_target_specific	fines_frac	0.047445	0.051733	0.002168	0.240072	0.019887
2	v1_target_specific	oversize_frac	0.432358	0.224458	0.045259	0.801378	0.416259
3	v1_target_specific	R95	288.544250	230.479605	48.541924	945.481917	171.983077
4	v1_target_specific	R50_fines	323.398563	226.841124	63.706866	825.765013	207.213529
5	v1_target_specific	R50_oversize	142.488865	105.015312	27.498061	406.201873	88.196924
6	v2_hybrid_extra_trees	P80	157.708072	33.479974	76.016131	230.519576	153.137936

7	v2_hybrid_extra_trees	fines_frac	0.050271	0.059135	0.001579	0.
	↪265903	0.017260				
8	v2_hybrid_extra_trees	oversize_frac	0.468492	0.210192	0.033226	0.
	↪847572	0.437667				
9	v2_hybrid_extra_trees	R95	298.530443	241.158254	48.776910	996.
	↪810828	177.453510				
10	v2_hybrid_extra_trees	R50_fines	330.556280	235.993585	64.210862	873.
	↪066616	213.098175				
11	v2_hybrid_extra_trees	R50_oversize	146.295647	108.429543	28.072745	414.
	↪330461	91.697836				
12	v2_best_models	P80	157.708072	33.479974	76.016131	230.
	↪519576	153.137936				
13	v2_best_models	fines_frac	0.050271	0.059135	0.001579	0.
	↪265903	0.017260				
14	v2_best_models	oversize_frac	0.422195	0.238924	0.030346	0.
	↪796151	0.400134				
15	v2_best_models	R95	298.530443	241.158254	48.776910	996.
	↪810828	177.453510				
16	v2_best_models	R50_fines	330.556280	235.993585	64.210862	873.
	↪066616	213.098175				
17	v2_best_models	R50_oversize	146.295647	108.429543	28.072745	414.
	↪330461	91.697836				

	target	mean_diff_best_minus_v1	median_abs_diff	max_abs_diff
0	P80	3.297491	3.365202	13.683074
1	fines_frac	0.002826	0.002045	0.044400
2	oversize_frac	-0.010163	0.018356	0.088960
3	R95	9.986193	7.163189	66.517917
4	R50_fines	7.157717	5.235717	47.301603
5	R50_oversize	3.806782	2.440697	27.651072

1.11 11. Save model and metadata

```
[13]: model_path = MODELS_DIR / "final_forward_v2_best_models_pipeline.joblib"
      joblib.dump(final_model, model_path)

      metadata = {
          "model_name": "TargetSpecificPipelineModel",
          "final_config_name": FINAL_CONFIG_NAME,
          "target_columns": target_cols,
          "input_columns": input_cols,
          "target_config": {
              target: {
                  "model": cfg["model"],
                  "feature_version": cfg["feature_version"],
                  "n_features": len(cfg["features"]),
                  "features": cfg["features"],
```

```

    }
    for target, cfg in FINAL_CONFIG.items()
},
"submission_path": str(submission_path),
"model_path": str(model_path),
"constraints": output_constraints,
}
metadata_path = MODELS_DIR / "final_forward_v2_best_models_metadata.json"
with open(metadata_path, "w") as f:
    json.dump(metadata, f, indent=2)

print("Saved model to:", model_path)
print("Saved metadata to:", metadata_path)

```

Saved model to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_forward_v2_best_models_pipeline.joblib
 Saved metadata to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_forward_v2_best_models_metadata.json

1.12 Final decision notes

Use this notebook to decide between:

outputs/submissions/prediction_submission_final_target_specific.csv	# v1
outputs/submissions/prediction_submission_forward_v2_hybrid.csv	# v2 sklearn ExtraTr
outputs/submissions/prediction_submission_forward_v2_best_models.csv	# v2 best models ref

For competition use, prefer the version that improves cross-validation metrics without creating suspicious shifts in test prediction distributions.